Exploiting Hybrid Energy Storage to Minimize the Carbon Footprint of AI Data Centers

Simon Wu
Ohio State University
Columbus, USA
wu.6540@buckeyemail.osu.edu

Xiaorui Wang
Ohio State University
Columbus, USA
wang.3596@osu.edu

Abstract

Big tech companies are rapidly deploying large GPU data centers and pursuing 24/7 carbon-neutral operation with wind and solar. Yet renewables are intermittent, AI workloads are highly variable, and inference is non-deferrable under strict SLOs, causing persistent supply–demand mismatches.

We propose *HES4AI*, a hybrid energy storage framework to minimize the carbon footprint of AI data centers that run ML training and inference on different timescales. *HES4AI* consists of three major components: (1) a three-tier energy-storage hierarchy that captures fast wind fluctuations with supercapacitors (SCs) while using uninterrupted power supply (UPS) batteries and thermal energy storage (TES) to shift energy over longer horizons and couple IT and cooling; (2) a two-stage *pair-then-pool* routing policy that pairs renewables with loads and storage first and pools only residual imbalances, reducing the required rating and CapEx of the shared pooling interface; and (3) an online constrained linear program that jointly optimizes routing, storage dispatch, and bounded training elasticity under practical state of charge (SoC) and power limits.

On a hardware testbed with Nvidia GPUs and a Maxwell supercapacitor, *UPS-only* consumes **1.38–1.42**× more brown energy and wastes **1.87–1.93**× more renewables than *HES4AI*. Across four regions in trace-driven simulations, *HES4AI* outperforms the conventional micro-grid, two UPS-only state of the arts methods, and a heuristic method under the same storage budget and *PTP*-sized pooling cap: Relative to *HES4AI* (1.0), baselines use **1.21–2.51**× more brown energy and waste **2.42–2.51**× more renewable energy.

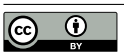
CCS Concepts

• **Computing methodologies** → **Modeling and simulation**; • **Hardware** → **Impact on the environment**; **Renewable energy**; **Enterprise level and data centers power issues**; **Batteries**.

Keywords

Data center, AI, hybrid energy storage, renewable energy

ACM Reference Format:
Simon Wu and Xiaorui Wang. 2026. Exploiting Hybrid Energy Storage to Minimize the Carbon Footprint of AI Data Centers. In *2026 International Conference on Supercomputing (ICS '26), July 06–09, 2026, Belfast, United Kingdom.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3797905.3800552



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICS '26, Belfast, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2522-7/26/07
https://doi.org/10.1145/3797905.3800552

1 Introduction

The rapid development of artificial intelligence (AI) has driven an increasing demand for computational power. So far, AI computation heavily relies on GPUs due to their unmatched computational efficiency in parallel processing. As an arms race, major AI companies (e.g., OpenAI, Meta, Microsoft, Google) are all rushing to purchase or build more GPU chips to significantly enhance their AI infrastructures [6]. As a result, Nvidia recently reported a record-high revenue of $26.0 billion for the first quarter of 2025, which is up 262% from a year ago [5]. Hence, it is reasonable to anticipate that the number of GPUs used in the data centers of those Big Tech companies for AI computation will increase sharply in the next few years.

In addition to high electricity bills, the enormous energy consumption of GPU data centers can also lead to negative environmental impacts (e.g., CO_2 emissions and global warming), due to their large carbon footprints. The reason is that most electricity produced worldwide comes from carbon-intensive sources, e.g., coal-fired generation [29]. Such energy, produced using conventional fossil-based fuels, is commonly referred to as brown energy. It has been reported that Google's carbon emissions went up 48% over the last five years due to AI [1, 3]. In contrast to brown energy, green energy is generated from renewable sources, such as wind turbines and solar panels, and is thus more environmentally sustainable. Therefore, to mitigate the negative environmental implications caused by their rapidly increasing energy consumption, many Big Tech companies have started taking various initiatives to operate their data centers on renewable (or green) energy. For example, Microsoft, Meta, and Google all have plans to become net-zero by 2030, while Amazon is targeting 2040 due to its logistics-heavy business [2, 4, 46].

There are several major challenges in exploiting renewable energy to power GPU data centers for AI development. First, renewable generation is inherently intermittent: wind and solar output depend on weather conditions and can vary substantially over time, leading to frequent supply shortfalls. Second, AI workloads in data centers are highly time-varying [38, 52]. For example, the request volume of *ChatGPT* can change dramatically over short periods, creating sharp power ramps and bursty demand spikes. Third, a common way to operate under limited energy supply is to throttle, defer, or shed computation [19, 30, 31]. However, this approach is poorly suited to ML/AI services because inference requests are non-deferrable under stringent service-level objectives (SLOs). Finally, while uninterrupted power supply (UPS) batteries are widely deployed and can buffer renewable variability [11, 31], their limited charge/discharge power and cycle-life degradation under frequent

cycling make them ineffective at absorbing fast renewable fluctuations or serving short-duration, high-power transients, especially those driven by bursty inference clusters.

In this paper, we propose *HES4AI*, a hybrid energy storage framework that minimizes the carbon footprint of AI data centers running ML training and inference on different timescales. In prior work, supercapacitors (SCs) and thermal energy storage (TES) have mainly been used for peak shaving, i.e., discharging stored energy to reduce peak power consumption [35, 55, 57, 59]. They are typically not treated as renewable buffers that absorb surplus generation and later release it to increase renewable utilization and reduce brown energy dependence. To the best of our knowledge, *HES4AI* is the first study to jointly leverage SCs and TES together with UPS batteries for renewable buffering across sub-minute, minute-to-hour, and multi-hour timescales, while explicitly coupling compute and cooling. This hybrid buffering view is critical for AI data centers because both the renewable supply and the IT/cooling demand exhibit variability in timescales.

HES4AI is motivated by the heterogeneity of AI data center workloads and renewable sources: inference jobs are unpredictable, bursty and non-deferrable under strict SLOs, whereas training is elastic; meanwhile, wind typically more variable and no general daily pattern compared to solar. We therefore introduce a two-stage *pair-then-pool* (*PTP*) routing policy. In *Stage 1*, when both workloads are active, *PTP* pairs sources with loads: solar serves training and charges the UPS/TES, while wind serves inference and charges the SC/UPS, so most renewable energy is absorbed along source-paired paths. In *Stage 2*, only the residual renewables are sent to a shared pooling interface of the micro-grid that can serve either workload and admits grid (brown energy) as backup. By pooling only residual imbalances rather than pooling all renewables upfront, *PTP* reduces the required power rating and the capital expense (CapEx) of the shared interface. At runtime, *HES4AI* solves a constrained linear program (LP) that jointly decides two-stage routing, hybrid energy storage devices (ESDs) dispatch, and bounded training elasticity under practical SoC limits, charging/discharging power ratings, and operational charge/discharge window constraints, to minimize brown energy usage and renewable wasted.

Specifically, this paper makes the following contributions:

- **SC+UPS+TES storage architecture for timescale-aware renewable buffering.** We design a three-tier storage hierarchy that buffers fast wind fluctuations with SCs, leverages UPS batteries for minute-to-hour balancing, and uses TES to shift renewable energy across multi-hour cooling demand, explicitly coupling IT and cooling.
- **PTP routing vs. conventional micro-grid pooling.** We propose a two-stage *pair-then-pool* policy that pairs renewables with different workloads and energy storage devices first and pools only residual imbalances through a shared micro-grid interface, reducing the required interface power rating and cost compared to fully pooled designs.
- **Online optimization with practical constraints.** We develop an online constrained LP that jointly optimizes routing, storage dispatch, and bounded training elasticity under SoC bounds, charging/discharging power limits, and operational window constraints to minimize brown energy and renewable curtailment.

The remainder of the paper is organized as follows. Section 2 provides background. Section 3 presents the *HES4AI* architecture, the *PTP* routing policy, and our online rolling-horizon LP controller with a feasible fallback. Section 4 reports the hardware testbed and results. Section 5 evaluates *HES4AI* at scale using trace-driven simulations and ablations. Section 6 surveys related work, and Section 7 concludes.

2 Background

This section summarizes (i) how GPU-intensive AI data centers and their training vs. inference workloads differ from traditional CPU-centric facilities, (ii) why renewable variability at different timescales motivates a multi-tier buffering design, and (iii) the storage technologies (SC, UPS, TES) that support compute–cooling coupled dispatch.

2.1 AI Data Centers vs. Traditional CPU-based Data Centers

Traditional data centers predominantly run CPU-bound workloads with relatively stable power usage. In contrast, AI data centers rely heavily on GPUs, which can exhibit much higher and more variable instantaneous power draw due to their parallel processing capabilities. AI workloads can be broadly categorized into two main types: *training* and *inference*, each posing distinct demands on power and scheduling.

Training workloads typically involve building machine learning models (e.g., training deep neural networks for natural language processing, such as OpenAI's GPT family [13]). These jobs are extremely resource-intensive, can run from hours to weeks, and are usually scheduled in advance. This flexibility allows scheduling decisions to leverage predictable renewable patterns (e.g., solar's diurnal cycle).

Inference workloads consist of real-time user requests, often referred to as inference tasks. These jobs are latency-sensitive and unpredictable, requiring burst-capable power delivery to avoid violating strict service level objectives (SLOs). As such, inference workloads benefit most from energy sources and storage devices that rapidly respond to short-term demand spikes.

Because training and inference differ so markedly in both scheduling flexibility and power characteristics, many data center operators run them on separate GPU clusters. This arrangement prevents the need to halt training jobs when inference experiences sudden demand surges [7, 8, 24].

2.2 Hybrid Renewable Energy System

AI data centers often incorporate multiple renewable sources to reduce the carbon-intensive grid reliance, thereby reducing their operational carbon emissions. In this work, we target solar and wind power, which are both more flexible in siting than hydropower or tidal power and widely adopted by data center operators [9, 10]. Figure 1 shows weekly solar and wind power traces for four representative locations. Solar generation follows a consistent diurnal pattern and wind exhibits characteristic intermittency, both profiles still display distinct shapes and magnitudes across regions.

Solar power exhibits a predictable diurnal cycle, peaking around midday and tapering in the evening [21]. While clouds and seasons introduce variability, solar is generally more stable over short

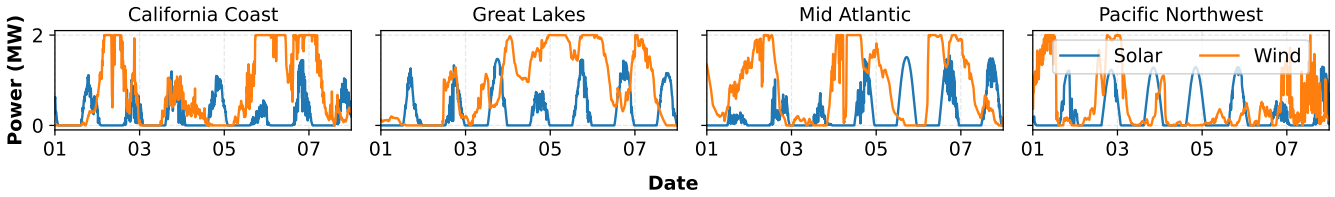
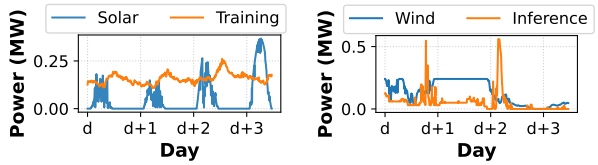


Figure 1: Weekly solar and wind power traces from four representative locations. Wind is per-turbine electrical output derived from wind-speed telemetry using a 2 MW turbine power curve (see Section 5.1). [16, 17, 27, 37]



(a) Example of solar power generation and training power trace

(b) Example of wind power generation and inference power trace

Figure 2: Solar and wind traces shown alongside training and inference power traces (illustrative). Workload traces are derived from [50].

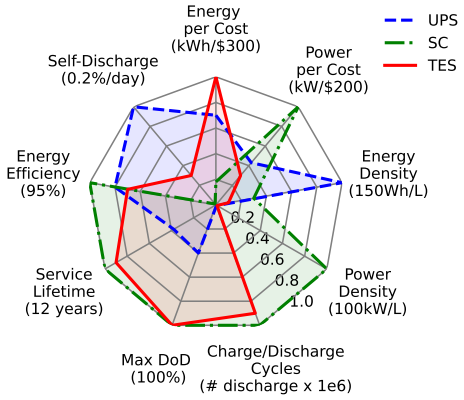


Figure 3: Comparison of UPS batteries, supercapacitors (SC), and TES along key metrics.

windows than wind, making it well suited for training workloads that can be scheduled in advance (Figure 2a). Also, surplus energy during peak hours can be stored for later use, reducing reliance on brown energy during off-peak times.

Wind power exhibits highly irregular up-and-down swings driven by gusts and local turbulence. Figure 2b illustrates that wind output lacks a stable daily pattern and can fluctuate sharply within short windows; meanwhile, inference demand is non-deferrable and can exhibit abrupt bursts under strict SLOs. As a result, relying on instantaneous wind output to cover inference bursts is unreliable. We therefore use fast-response storage to absorb short-lived wind surpluses and to supply power during inference bursts, effectively shaping wind energy at sub-minute timescales.

2.3 Hybrid Energy Storage System

To buffer intermittent renewables and bursty AI workloads, AI data centers integrate energy storage technologies to stabilize power availability [9, 10]. In addition to server power, cooling systems account for a substantial portion of an AI data center's energy usage, often 30–40%, which means addressing cooling power is equally critical [55]. We incorporate three complementary storage technologies: SCs, UPS batteries, and TES. Figure 3 compares their characteristics across key metrics such as power density, response time, and energy retention [32, 36, 39, 56, 57, 59]. Values are normalized to the maximum across the three technologies; we plot energy/power per dollar and the inverse of self-discharge, so larger is better.

Supercapacitor (SC) is outstanding at handling brief and high-intensity power demands, benefiting from its high power density and fast charge/discharge capabilities [35, 59]. SCs feature low internal resistance, allowing it to deliver large bursts of current with minimal heat generation. This characteristic makes SCs ideal for workloads with short, unpredictable power spikes, such as latency-sensitive inference tasks. However, SCs exhibit higher self-discharge rates, which makes it less suitable for long-term energy storage.

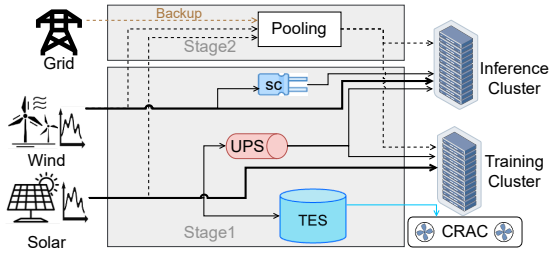
Uninterruptible Power Supply (UPS) can supply stable power for extended periods, making it well-suited to workloads that require continuous electricity for longer periods [35, 59]. In addition, its low self-discharge rate ensures that stored energy remains available for base load in inference demand. Additionally, UPS batteries have relatively high energy density, reducing the physical footprint in AI data centers. Nevertheless, chemical wear and depth-of-discharge (DoD) constraints impose practical limits on how frequently the battery can cycle without shortening its lifespan [18].

Thermal Energy Storage (TES) specifically tackles this cooling load by storing chilled water in insulated tanks. When renewable power is abundant, chillers can be run to chill the water. Later, when cooling demand surges with increases in workload, the chilled water is circulated to the computer room air conditioning (CRAC) units, reducing the immediate need to run power-intensive chillers.

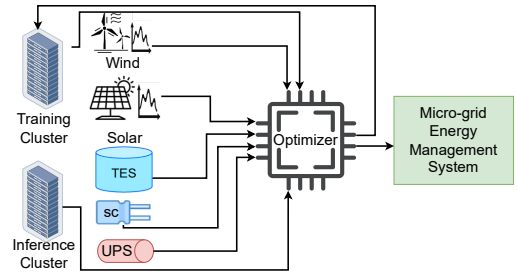
3 Design of *HES4AI*

3.1 System Overview and Control Loop

Figure 4 summarizes the overall *HES4AI* power and data flow. Figure 4a shows the physical power paths. *HES4AI* implements a two-stage renewable allocation policy rather than immediately pooling all sources into a single pooled resource. In *Stage 1* (solid arrows), solar is routed to the training cluster and to charge UPS batteries and TES, while wind is routed to the inference cluster and to charge SCs and UPS. This design is based on the real-world wind and solar power traces that show wind has more small-timescale fluctuations than solar (Figure 1). TES shifts cooling over multi-hour horizons



(a) System power flow with two-stage renewable routing: Stage 1 uses source-specific paths with local storage charging; Stage 2 exchanges only residual renewable energy via a shared pooling interface with grid backup.



(b) System data and control flow. The Optimizer receives renewable telemetry/forecasts, workload demands, and ESD SoC, issues renewable-allocation and ESD-dispatch commands via the microgrid EMS, and updates the training elasticity control.

Figure 4: *HES4AI* architecture. (a) Physical power paths and microgrid routing among renewables, grid, hybrid storage, and cooling. (b) Data and control flow: the Optimizer receives telemetry/forecasts and issues renewable allocation and ESD dispatch commands via the microgrid EMS and elastic constraints of training jobs.

via chilled water to the CRAC units, while SC discharges to cover fast inference transients and UPS discharges to support IT demand under device limits. In *Stage 2* (dashed arrows), we pool only the residual renewable power that cannot be consumed or stored in *Stage 1*. This residual is injected into a shared interface that can supply either cluster. The grid injects backup power through this pooling interface when residual renewables cannot meet IT demand. Accordingly, we cap the shared interface to $P_{\text{pool}}^{\max,\text{PTP}}$ (Section 3.3).

Figure 4b illustrates the corresponding data and control loop. Here, the *Optimizer* is a software controller that runs above the micro-grid Energy Management System (EMS) and computes per-interval dispatch decisions. At each control interval, the *Optimizer* receives (i) current solar and wind output together with short-horizon forecasts, (ii) instantaneous power requests from the training and inference clusters, and (iii) the SoC of ESDs and charge and discharge window flags for UPS and TES. Training workloads are elastic: their power can be slightly accelerated or slowed within a bounded range, controlled by a scheduling factor α, while inference demand remains non-deferrable. Based on these inputs, the *Optimizer* solves the rolling-horizon LP in Section 3.4.1 (Section 3.4.2) to produce per-interval routing, storage-dispatch, and *training-elasticity* decisions; on infeasibility or missing telemetry, it falls back to a lightweight rule-based controller.

3.2 Hybrid Storage Architecture and CapEx-based Configurations

Hybrid Energy Storage System Infrastructure. A conventional UPS-only architecture places a single battery tier in front of the IT and cooling loads. This UPS tier is used as a buffer for intermittent renewables, charging when renewable output is high and discharging when renewable output is low and grid power is expensive or carbon-intensive [11, 22, 31]. However, typical UPS batteries have a short design life (on the order of ~4 years) and are highly sensitive to frequent, deep cycling. To preserve reliability, operators typically over-provision UPS capacity to limit depth-of-discharge and keep sufficient backup margin. Meanwhile, frequent cycling accelerates aging and can shorten replacement intervals, which increases both upfront CapEx and total cost of ownership (TCO). Overall, a *UPS-only* design places nearly all short- and medium-timescale buffering burden on a single electrochemical tier, making it an expensive and

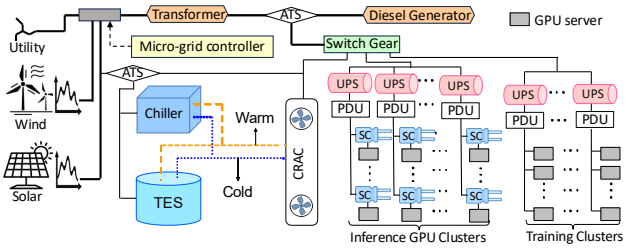


Figure 5: Proposed *HES4AI* system infrastructure with per-server SCs, PDU-level UPS, and facility-level TES.

durability-limited approach. To address these limitations, we replace the *UPS-only* buffer with a three-tier hybrid energy storage stack. Figure 5 shows the resulting physical placement and power paths in the AI data center.

In this infrastructure, on-site solar and wind connect to the facility switchgear, and an ATS transfers the bus between the utility interconnection and the standby diesel generator during grid disturbances. Downstream, we employ a distributed UPS design [28, 47] where each PDU-level UPS battery sits upstream of its local PDU and buffers the corresponding cluster's IT load (training or inference). Unlike prior work HEB [35] that places SCs at the rack level for peak power shaving, we place per-server SCs in the inference cluster to buffer fast, local inference spikes with minimal energy capacity. The cooling subsystem (chillers/TES/CRAC) is served from dedicated feeders, where TES is charged by running chillers during periods of high renewable availability.

Per-server SCs absorb sub-minute inference spikes locally, reducing high-frequency cycling seen by upstream components. PDU-level UPS batteries provide minute-to-hour buffering for IT demand while following practical charge/discharge windows. TES time-shifts cooling over multi-hour horizons by storing renewable energy as chilled water. This division of labor allows each tier to operate where it is most cost-effective and durable, rather than forcing a single UPS tier to handle all variability.

UPS-only vs. Hybrid Configurations. With the placement fixed, we run a trace-driven offline sizing study to compare a UPS-only baseline against a lower-CapEx hybrid ESD design under the same

sizing window. We parameterize the UPS-only sizing target by a fraction $f \in [0.1, 1.0]$, which specifies the fraction of a representative day's *net-deficit energy* that the UPS is provisioned to cover as usable energy. To focus sizing on challenging renewable conditions, we select a contiguous 30-day window from the annual trace after an initial warm-up period. To avoid overfitting, this sizing window is disjoint from the full-trace horizon used later in Section 5.

UPS-only baseline. For each $f \in [0.1, 1.0]$, we pick a representative day within the sizing window with non-trivial IT energy and compute its *net-deficit energy*: over the day, we sum the portion of demand not covered by available renewables. This quantity represents the usable energy budget needed for a charge-once-per-day UPS to cover that day's deficits. We then size the UPS usable energy to f times this net-deficit energy, and convert it to nameplate capacity using depth-of-discharge and round-trip efficiency assumptions. We derive the UPS charge/discharge power rating from a 4-hour C-rate so the power limit scales with the sized energy capacity. With the resulting fixed UPS configuration, we solve an LP dispatch over the full multi-day sizing window to minimize grid draw and wasted renewables subject to UPS SoC bounds, power limits, and efficiency, and record the resulting grid energy and wasted renewables as the baseline targets.

Hybrid sizing under grid/renewables targets. On the same sizing window, we solve a joint sizing-and-dispatch LP for the hybrid stack. The LP selects the energy capacities of ESDs and their dispatch schedules, and ties each tier's charge/discharge power limit to its sized energy capacity through a fixed C-rate. It enforces the architecture-specific charging/discharging paths in Section 3.1, and all tiers are charged only from renewables. To ensure cooling-side feasibility, TES energy and discharge power are capped by a cooling design envelope derived from the window's cooling demand (coverage-hours for capacity and a design cooling-power level for discharge rate). The objective minimizes total storage CapEx while requiring total grid energy and total wasted renewbales on the sizing window to be no worse than the UPS-only baseline. We use representative device cost and lifetime parameters from prior work [47, 57].

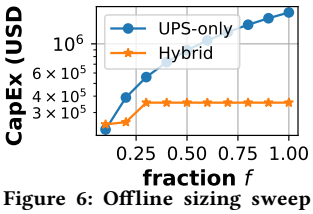


Figure 6: Offline sizing sweep ($f \in [0.1, 1.0]$): UPS-only CapEx grows with f while Hybrid quickly plateaus.

Table 1: Key points from sizing sweep.

f	UPS-only	Hybrid	Save
0.1	\$222.7k	\$243.8k	-9.4%
0.3	**\$558.5k**	**\$356.9k**	**36.0%**
0.5	\$894.3k	\$356.9k	60.1%
0.8	\$1.398M	\$356.9k	74.5%
1.0	\$1.733M	\$356.9k	79.4%

Results and insight. Figure 6 shows a clear divergence between the two architectures as we sweep $f \in [0.1, 1.0]$. The *UPS-only* baseline cost grows steeply with f, because a larger target swing directly translates into a larger battery energy capacity. In contrast, the hybrid design increases only slightly for small f and quickly enters a plateau (around $f \approx 0.3$), after which the optimizer repeatedly selects an essentially unchanged cost-minimizing configuration (Table 1). Once the UPS-only baseline targets stop improving with larger f, the hybrid problem is no longer asked to achieve anything stricter, so its optimal configuration stays essentially unchanged. The optimizer therefore stabilizes on a small, high-power/low-energy SC tier for short bursts, while relying on lower-cost UPS energy capacity and TES cooling shifting for longer horizons. Accordingly, in Section 5 we use a representative capacity configuration from the plateau regime and keep it fixed for all runtime evaluations.

3.3 Renewable Allocation and Micro-grid Sizing

Building on Sections 2.2 and 2.3, we ask whether the micro-grid must always provision a fully-pooled interface that aggregates all renewable generation before dispatch. Instead, we quantify how the required pooling-interface rating changes when the two-stage routing policy in Section 3.1 is used so that the shared pooling interface carries only residual transfers.

To quantify the provisioning benefit, we build a trace-driven offline LP and compare two dispatch policies that use the same renewable traces and loads but imply different required pooling-interface ratings.

Conventional fully-pooled facility design. We model the pooling interface as the first-stage aggregation bus that all renewable generation must traverse before any dispatch decisions are applied. Thus, the interface must be rated for the peak aggregated renewable power:

$$P_{\text{pool}}^{\max,\text{MG}} = \max_t \big(P_{\text{sol}}(t) + P_{\text{wind}}(t)\big),$$

and we model the corresponding CapEx as

$$\text{CapEx}_{\text{MG}} = C_{\text{MG}}^{P} \cdot P_{\text{pool}}^{\max,\text{MG}},$$

using per-kW micro-grid cost C_{MG}^{P} from prior work [48].

Pair-then-Pool (PTP) facility design. We formalize the two-stage routing policy in Section 3.1 for sizing: the shared pooling interface carries only residual renewable power after source-preferred local serving and storage charging decisions. Let $P_{\text{sol,pool}}(t)$ and $P_{\text{wind,pool}}(t)$ denote the residual solar/wind power injected into the pooled bus. The required pooling-interface rating is then

$$P_{\text{pool}}^{\max,\text{PTP}} = \max_t \big(P_{\text{sol,pool}}(t) + P_{\text{wind,pool}}(t)\big).$$

Figure 7 illustrates the key difference in what the shared pooling interface must carry under the two designs. With full pooling, the interface tracks the total renewable power $P_{\text{sol}}(t) + P_{\text{wind}}(t)$, whereas under *PTP* it sees only the Stage-2 residual $P_{\text{sol,pool}}(t) + P_{\text{wind,pool}}(t)$. Accordingly, the dashed lines mark the corresponding required ratings $P_{\text{pool}}^{\max,\text{MG}}$ and $P_{\text{pool}}^{\max,\text{PTP}}$. Under the linear per-kW cost model above, this directly translates into a proportional reduction in pooling CapEx. We use this offline facility-sizing result to fix $P_{\text{pool}}^{\max,\text{PTP}}$; Section 5 then evaluates runtime grid energy usage and renewable curtailment under this fixed cap.

3.4 Online Optimization Controller

3.4.1 Optimization Formulation. Having fixed the pooling-interface rating $P_{\text{pool}}^{\max,\text{PTP}}$ at design time (Section 3.3), we now formulate an online LP controller that dispatches renewables, storage, and training elasticity subject to this cap and device operating limits.

Notation. We discretize time into control intervals of length Δt

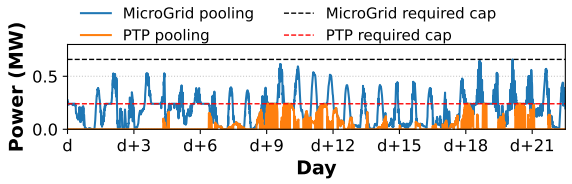


Figure 7: Full pooling vs. PTP pooling-interface rating. PTP forwards only Stage-2 residual renewables to the shared bus; dashed lines show $P_{\text{pool}}^{\max,\text{MG}}$ and $P_{\text{pool}}^{\max,\text{PTP}}$.

hours (e.g., $\Delta t = 5/60$ for 5-minute control). All power variables are in kW and energy states are in kWh. For each interval t, let $P_{\text{sol}}(t)$ and $P_{\text{wind}}(t)$ denote the realized solar and wind availability. Let $D_{\text{train}}(t)$ and $D_{\text{inf}}(t)$ denote the instantaneous electrical power demand of training and inference. Let $Q_{\text{cool}}(t)$ denote the cooling thermal load; we convert it to an equivalent electrical cooling demand as:

$$D_{\text{cool}}(t) = \frac{Q_{\text{cool}}(t)}{\text{COP}_{\text{CRAC}}}, \tag{1}$$

where the empirical COP model [12] is a function of the supplied air temperature T_{sup}:

$$\text{COP}^{\text{CRAC}} = 0.0068\,T_{\text{sup}}^2 + 0.0008\,T_{\text{sup}} + 0.458.$$

Assuming $T_{\text{sup}} = 25°\text{C}$, we obtain $\text{COP}^{\text{CRAC}} \approx 4.7$.

Table 2 summarizes the key per-interval decision variables (all powers in kW). SC/UPS/TES energy states are tracked separately in kWh and updated across intervals.

Description	Variable(s)
Stage 1 solar used for training (direct)	$P_{\text{sol,train}}(t)$
Stage 1 wind used for inference (direct)	$P_{\text{wind,inf}}(t)$
SC charge / discharge	$P_{\text{sc,c}}(t)$, $P_{\text{sc,d}}(t)$
UPS charge from solar / wind (*Stage 1*)	$P_{\text{ups,sol}}(t)$, $P_{\text{ups,wind}}(t)$
UPS discharge to loads	$P_{\text{ups,train}}(t)$, $P_{\text{ups,inf}}(t)$
TES charge / discharge (solar-side *Stage 1*)	$P_{\text{tes,c}}(t)$, $P_{\text{tes,d}}(t)$
Stage 2 residual injected into pooled bus	$P_{\text{sol,pool}}(t)$, $P_{\text{wind,pool}}(t)$
Stage 2 pooled-bus supply to IT loads	$P_{\text{pool,train}}(t)$, $P_{\text{pool,inf}}(t)$
Grid injection into pooled bus (IT backup)	$G_{\text{pool}}(t)$
Grid power to cooling load	$G_{\text{cool}}(t)$
Training elasticity	$\alpha(t)$
Curtailment	$W_{\text{sol}}(t)$, $W_{\text{wind}}(t)$

Table 2: Key per-interval decision variables for *HES4AI*. All listed quantities are powers (kW); SC/UPS/TES energy states are tracked in kWh across intervals.

Training is elastic: the cluster can vary its active training level over time (e.g., pausing low-priority jobs, adjusting power caps). We model this with a scaling factor $\alpha(t) \in [\alpha_{\min}, \alpha_{\max}]$ on the baseline training-power trace: $\alpha(t) < 1$ defers work (backlog grows) and $\alpha(t) > 1$ catches up when renewable supply is abundant. Policy and job constraints limit elasticity, captured by the bounds on $\alpha(t)$ and the work-completion normalization in Section 5.1.

Objective. *HES4AI* minimizes grid energy usage and renewable curtailment, while using training elasticity to better align consumption with renewable availability. For each interval t, we define

$$total_grid(t) = G_{\text{pool}}(t) + G_{\text{cool}}(t), \tag{2}$$

$$wasted_term(t) = W_{\text{sol}}(t) + W_{\text{wind}}(t). \tag{3}$$

We introduce an elasticity reward term $scale_term(t)$. Let $P_{\text{sol}}^{f}(t)$ and $P_{\text{wind}}^{f}(t)$ denote short-horizon forecasts of solar and wind availability over the planning horizon. Forecasts are used only in the objective to shape $\alpha(t)$; all power-balance and device constraints are enforced using realized renewable availability.

When both training and inference are active, *Stage 1* pairs solar with the elastic training workload, so we define

$$scale_term(t) = \left(P_{\text{sol}}^{f}(t) - P_{\text{sol}}(t)\right)\left(1 - \alpha(t)\right), \tag{4}$$

with the sign chosen to reward $\alpha(t) > 1$ when current solar exceeds the forecasted future level, and reward $\alpha(t) < 1$ when solar is expected to improve later. Wind primarily serves the non-deferrable inference load in this regime and thus does not directly drive $\alpha(t)$. When only training is active, we compute the reward using pooled renewable forecasts $P_{\text{pool}}^{f}(t) = P_{\text{sol}}^{f}(t) + P_{\text{wind}}^{f}(t)$ and realized availability $P_{\text{pool}}(t) = P_{\text{sol}}(t) + P_{\text{wind}}(t)$. For intervals where training is inactive, we set $scale_term(t) = 0$.

Since forecasts are exogenous parameters and $\alpha(t)$ enters linearly, the objective remains linear. Over the horizon, we minimize

$$\min \sum_t \Big(total_grid(t) + \lambda_w\, wasted_term(t) - \lambda_\alpha\, scale_term(t) \Big). \tag{5}$$

where λ_w and λ_α are weights that normalize curtailment and elasticity reward relative to grid power.

Per-interval power balance. At each interval t, we enforce power balance at the renewable sources, the pooled interface, and each load. All power-flow variables (including curtailment and grid injections) are constrained to be non-negative. Importantly, all ESD charging decisions occur in *Stage 1* (i.e., before pooling), so storage charging variables appear only in the source-side balances below:

$$P_{\text{sol}}(t) = P_{\text{sol,train}}(t) + P_{\text{ups,sol}}(t) + P_{\text{tes,c}}(t) + P_{\text{sol,pool}}(t) + W_{\text{sol}}(t). \tag{6}$$

$$P_{\text{wind}}(t) = P_{\text{wind,inf}}(t) + P_{\text{sc,c}}(t) + P_{\text{ups,wind}}(t) + P_{\text{wind,pool}}(t) + W_{\text{wind}}(t). \tag{7}$$

$$P_{\text{pool,train}}(t) + P_{\text{pool,inf}}(t) = P_{\text{sol,pool}}(t) + P_{\text{wind,pool}}(t) + G_{\text{pool}}(t). \tag{8}$$

$$P_{\text{sol,train}}(t) + P_{\text{ups,train}}(t) + P_{\text{pool,train}}(t) = \alpha(t)\, D_{\text{train}}(t). \tag{9}$$

$$P_{\text{wind,inf}}(t) + P_{\text{sc,d}}(t) + P_{\text{ups,inf}}(t) + P_{\text{pool,inf}}(t) = D_{\text{inf}}(t). \tag{10}$$

$$P_{\text{tes,d}}(t) + G_{\text{cool}}(t) = D_{\text{cool}}(t). \tag{11}$$

$$P_{\text{sol,pool}}(t) + P_{\text{wind,pool}}(t) + G_{\text{pool}}(t) \le P_{\text{pool}}^{\max,\text{PTP}}. \tag{12}$$

Two-stage renewables routing. We encode the two-stage routing policy from Section 3.1 directly in the source-side power-balance constraints (Eq. 6–7) and the pooling-interface capacity cap (Eq. 12). When both training and inference are active, cross-use is disabled by construction because solar can only be allocated to the training-side paths ($P_{\text{sol,train}}, P_{\text{ups,sol}}, P_{\text{tes,c}}, P_{\text{sol,pool}}$) and wind can only be allocated to the inference-side paths ($P_{\text{wind,inf}}, P_{\text{sc,c}}, P_{\text{ups,wind}}, P_{\text{wind,pool}}$). When at least one workload is idle, we relax the pairing rule by allowing a larger fraction of renewables to enter the shared pooling interface to serve the active workload, thereby reducing curtailment.

Device power ratings and charge/discharge windows. UPS and TES charging/discharging powers are limited by their nameplate ratings. We capture operational practices using exogenous

charge/discharge window flags, rather than optimizing binary decisions inside the LP. Specifically, we use four per-interval indicators $u_c^{ups}(t), u_d^{ups}(t), u_c^{tes}(t), u_d^{tes}(t) \in \{0,1\}$, which specify whether the UPS/TES is allowed to charge or discharge at time t. When a charging (or discharging) flag is inactive, the corresponding charging (or discharging) power is forced to zero; when active, it is capped by the device's rated charge/discharge power. We also enforce that each device cannot charge and discharge in the same interval (i.e., the charge and discharge windows are mutually exclusive).

By default, we allow UPS discharge whenever needed but restrict UPS charging to follow a common *daily charge-once* practice: each day contains at most one contiguous UPS charging block (e.g., one hour), which limits frequent cycling and better reflects UPS management policies. For TES, we adopt a multi-hour charging window (e.g., six hours) to represent chilled-water pre-cooling operations, and allow TES to discharge outside that window. SCs do not use window flags and may charge/discharge in any interval subject only to their power ratings and SoC bounds. For fairness, any baseline that uses UPS/TES is evaluated under the same window-flag parameters as *HES4AI*.

ESD SoC energy balance. Across intervals, we track the energy state of each ESD in kWh (electrical-equivalent). For example, the UPS energy state $E^{ups}(t)$ evolves as

$$\begin{aligned} E^{ups}(t{+}1) = (1-\rho^{ups})\,E^{ups}(t) \\ + \eta_c^{ups}\Big(P_{ups,sol}(t) + P_{ups,wind}(t)\Big)\,\Delta t \\ - \frac{P_{ups,train}(t) + P_{ups,inf}(t)}{\eta_d^{ups}}\,\Delta t. \end{aligned} \tag{13}$$

subject to $E_{min}^{ups} \le E^{ups}(t) \le E_{max}^{ups}$. Here, η_c^{ups} and η_d^{ups} are charge and discharge efficiencies, ρ^{ups} is the per-interval self-discharge fraction, and Δt is in hours.

SC and TES follow the same linear structure with their own efficiencies, self-discharge, and capacity bounds.

Training elasticity constraints. Training is elastic: at each interval we scale its instantaneous power by an elasticity factor $\alpha(t)$, while inference demand remains non-deferrable. We bound the elasticity factor as $\alpha_{min} \le \alpha(t) \le \alpha_{max}$ for all t. We do not impose additional cumulative training-work constraints over the look-ahead horizon. This keeps the online LP lightweight: the only cross-interval state coupling comes from the ESD SoC dynamics. Training-work fairness is enforced in post-processing by normalizing results to the same executed work (Section 5).

In our implementation, we do not enforce additional per-day or horizon-wide work-conservation constraints inside the LP. Instead, we record the realized per-interval $\alpha(t)$ decisions and normalize the reported results in Section 5 to ensure a fair comparison across methods. Specifically, we compute the cumulative executed training work $W(t) = \sum_{\tau \le t} \alpha(\tau)\, D_{train}(\tau)\, \Delta t$, choose a common cutoff time $T^\star$ at which all methods have executed (approximately) the same amount of training work as the most conservative run, and report grid-energy and curtailment metrics over the truncated horizon $[0, T^\star]$. This normalization ensures that elasticity changes *when* training is performed, not *how much* training is performed in the reported results.

3.4.2 Solution Method and Fallback Controller. At runtime, *HES4AI* runs as an online linear–programming controller that repeatedly solves the optimization problem from Section 3.4.1 using updated telemetry and forecasts. Storage devices follow the linear SoC models described earlier, so the overall problem remains a linear program with a linear objective and linear constraints.

Rolling-horizon LP. At each control interval, the Optimizer looks ahead for the next H control steps using the latest telemetry and renewable forecasts, solves an LP to plan renewable allocation, storage dispatch, and training elasticity over this look-ahead window, and then executes only the decision for the current step. At the next interval, it repeats the same procedure with refreshed telemetry/forecasts so decisions continuously adapt to changing conditions. If the LP is infeasible or the solver fails (e.g., due to missing telemetry or numerical issues), the system falls back to the rule-based controller described below, which preserves the same two-stage routing policy.

Complexity and practicality. The LP size grows linearly with the look-ahead length H because we introduce a constant number of decision variables per control step. With our default configuration (5-minute control and a horizon of H steps), each LP contains on the order of a few thousand continuous variables and constraints. In our trace-driven experiments, the LP solve time is well below one control interval on a commodity server, leaving ample slack for telemetry ingestion and actuation.

Fallback controller (optional). We implement a simple rule-based fallback that is triggered when the LP is infeasible or telemetry is unavailable. The fallback preserves the same two-stage routing policy (Section 3.1) and enforces the same device power limits and charge/discharge windows. It prioritizes SC for fast inference transients, uses UPS/TES to cover slower IT/cooling deficits within their admissible windows, and draws grid power only when renewables and available ESD discharge are insufficient.

4 Hardware Testbed Evaluation

Setup. Figure 8 shows the key components and connections of our hardware testbed. We evaluate on a server with two Nvidia GeForce RTX 3090 GPUs ((7); 350,W TDP each). GPU0 runs training (one of *ResNet-50* [25], *ViT-B/16* [15], *EfficientNet-B0* [45], or *MobileNetV3-Large* [26]); GPU1 runs a fixed *GPT-2-small* inference job and is never rescheduled. The server is connected to a scaled-down hybrid energy storage system (ESS): a Maxwell 16 V/500 F SC ((1); stepped to 12 VDC (2) and inverted to 110 VAC (3)) and a 1500 VA/900 W line-interactive UPS ((6); two 12 V, 9 Ah SLA cells in series; ≈216 Wh nominal). Power routing uses two cascaded automatic transfer switches ((4); ATS), enforcing priority *wall power* → *SC* → *UPS*; a downstream source engages only after the upstream source is disconnected. A Raspberry Pi (9) controls four relays (5) for SC and UPS charge and discharge gating and one relay for wall-power toggling, and collects telemetry at 10 Hz. We sample GPU power and utilization every 0.1,s via `nvidia-smi` and system power via ACPI. The monitor (8) is used to track progress. All power-limit set points and power traces are logged to CSV.

To emulate time-varying renewables while electrically powering the server from the wall, we replay a known 5-min renewables trace at 3000× speed (5 min → 0.1 s). At each 0.1 s tick, we hold the power-limit target associated with the current 5-min slot and

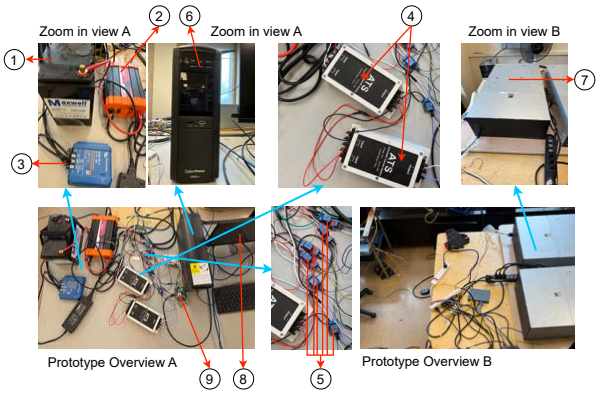


Figure 8: Hardware testbed setup

apply bounded changes to GPU0's power limit using `nvidia-smi -pl`. This aligns training demand with the renewable pattern, while GPU1's inference workload remains unchanged. The setup lets us compare *HES4AI* with *UPS-Only* under identical workloads and renewable conditions. To equalize usable energy across configurations, we set *UPS-Only* to 216 Wh as mentioned before at 80% DoD (172.8 Wh usable). In *HES4AI*, the SC contributes around 8 Wh, so we allocate the remaining around 165 Wh to the UPS. This keeps the total usable ESD budget the same for fairness.

Rescheduling mechanism. We simulate training rescheduling by modulating GPU0's power limit around a 200 W nominal set point. At the beginning of each 5-min slot, we compare the current renewable level to the average over a future window of 288 slots (one day; 288×5 min = 1440 min). If the current level exceeds the future average, we *pull forward* work by raising the power limit; otherwise, we *defer* by lowering the power limit. This power-limit schedule is the only control applied to training; inference is fixed.

ESD dispatch policy. Given the predefined renewable trace and the real-time server demand, the instantaneous surplus/deficit is computed as the difference between supply and demand. Under *HES4AI*: on surplus, energy first charges the SC, then the UPS; any remainder is counted as *Wasted*. On deficit, the SC discharges first, then the UPS; any remaining shortfall is served by the grid and counted as *Brown Energy*. The *UPS-Only* configuration follows the same logic with the SC disabled.

Metrics. We report: (i) *brown energy usage*—total grid energy consumed (integral of the gray *Brown Energy* area in Figure 9); and (ii) *green energy wasted*—renewable energy that could not be used or stored (integral of the burgundy *Wasted* area). Energy is computed by integrating the 0.1 s power traces and is reported in Watt-hours (Wh).

Figure 9 presents the time-series decomposition under the two storage configurations. The top panel (*HES4AI*) enables both SC and UPS; the bottom panel (*UPS-Only*) disables the SC. The y-axis shows instantaneous power (W). Black shows renewable supply, red shows rescheduled demand, and the green dashed curve shows the original demand. Shaded regions decompose how surplus/deficit is handled: On surplus we show *SC charge* and *UPS charge*, with any remainder labeled *Wasted*; on deficit we show *SC discharge*, *UPS discharge*, and *Brown Energy*. Relative to *UPS-Only*, the *HES4AI* panel exhibits smaller wasted-renewable (burgundy) and brown energy (gray) areas, indicating better surplus capture and lower grid reliance.

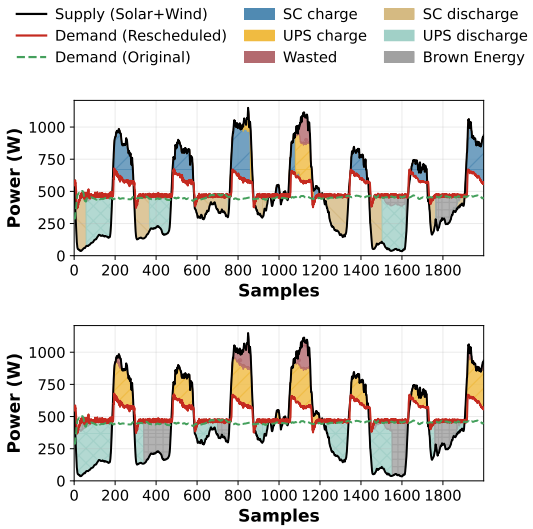
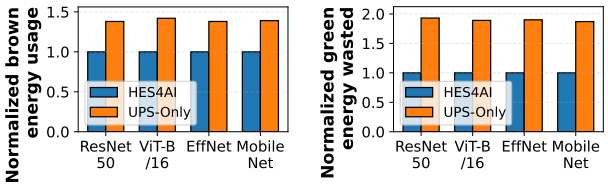


Figure 9: Time-series decomposition under two storage configurations

Figure 10 aggregates results across the four training models (each paired with the same *GPT-2-small* inference on the second GPU). Figure 10a plots normalized brown energy use (HES4AI = 1.0); *UPS-Only* requires 1.38×–1.42× more brown energy than *HES4AI*, showing that the SC helps cover renewable deficits and lowers grid draw. Figure 10b plots normalized wasted renewable; *UPS-Only* wastes 1.87×–1.93× more than *HES4AI*, indicating that the SC substantially increases surplus capture. The effect is consistent across models: adding a modest SC reduces grid dependence during deficits and curtails waste during surpluses.



(a) Relative brown energy used (b) Relative green energy wasted

Figure 10: Comparing the brown energy used and green energy wasted between *HES4AI* (top) and *UPS_Only* (bottom)

5 Simulation Evaluation

We evaluate *HES4AI* under realistic renewable conditions and AI workload traces, and compare it against four baselines. We first describe the simulation setup and the baselines. We then compare *HES4AI* against these baselines in terms of brown energy usage and green energy wasted, isolate the benefit of a hybrid ESDs under a fixed *PTP* architecture, and examine the effect of using hybrid renewables vs. single-source deployments. Finally, we perform sensitivity studies on the renewable supply–demand ratio, storage capacities, and forecast accuracy.

5.1 Simulation Setup

Renewable trace modeling. We derive 5-minute wind and solar power traces from public NREL datasets (see Figure 1). For wind, we use the WIND Toolkit NOW-23 data set for a representative sites and extract year-2019 wind speed at 100 m height [16, 17, 27].

Wind speed is converted to electrical power using an Enercon E82 2 MW type-4 turbine model based on the NREL/ESIG power curve, with cut-in/cut-out speeds of 4/28 m/s and a ramp-rate limit of roughly 10% of rated power per step to smooth high-frequency fluctuations [37].

For solar, we use the NSRDB GOES CONUS product for a nearby site, taking global horizontal irradiance (GHI) and ambient temperature at 5-minute resolution [40]. These irradiance and temperature series are fed into a simplified PVWatts v5 DC/AC model with temperature-dependent efficiency and a DC/AC ratio of 1.2, which lies within the typical range ($\approx$ 1.2–1.4) used for utility-scale PV systems [14].

We apply a single global scaling step to place renewables and workloads on a comparable power level: solar is scaled so its long-term average matches the long-term average training power, and wind is scaled so its long-term average matches the long-term average inference power over the aligned horizon. This scaling only matches long-term means (not per-interval pairing) and is used as a normalized default.

GPU workloads and power model. We adopt GPU-based training and inference traces from Alibaba's MLaaS PAI dataset [50], covering 1.5 months of production usage, and split them into training and inference clusters by task type (`worker` for training and `evaluator` for inference). Per-GPU power is modeled as $P_{\text{gpu}} = P_{idle} + (P_{peak} - P_{idle}) \times U_{ti}$, with $P_{idle} = 25$ W and $P_{peak} = 250$ W in our normalized per-node model. Summing P_{gpu} over all active GPUs yields the cluster-level series at 5-minute resolution. Cooling-side electricity is derived from IT heat using Equation 1 (Section 3.4.1).

Training elasticity. The bounds of elasticity factor are $\alpha_{\min} = 0.5$, $\alpha_{\max} = 1.5$. Because $\alpha(t)$ changes when training is executed, to ensure a fair comparison across methods we post-process all runs to a common cutoff time $T^\star$ such that each method has executed approximately the same cumulative training work, and report brown energy usage and green energy wasted metrics over $[0, T^\star]$.

ESD configuration and parameters. We use the representative SC+UPS+TES configuration from the design-time sizing study (Section 3.2) and keep device capacities fixed across all methods unless stated otherwise. We model SC/UPS/TES with SoC bounds, charge/discharge power limits, and round-trip efficiency. We assume self-discharge of 20% per day for SC, 0.2% per day for UPS, and 3% per day for TES; UPS depth-of-discharge is 80% [28], and TES operates between 5°C and 12°C. Unless otherwise stated, we enable the operational "daily charge-once" policy for UPS (and consistent windows for TES) for all methods that use these devices.

5.2 Baselines

Fairness policy. All methods are evaluated on total facility electricity (IT + cooling), where cooling power is always included via Equation 1. All methods share the same physical pooling-interface rating cap $P_{\text{pool}}^{\max,\text{PTP}}$, which is fixed by the design-time PTP sizing step (Section 3.3). For baselines with SC+UPS+TES, we use the same device capacities, SoC bounds, power limits, and charge/discharge windows as *HES4AI*. For the UPS-only baselines, we convert *HES4AI*'s total ESD CapEx into an equivalent UPS-only capacity so the aggregate storage budget matches. For training elasticity, we use the same α bounds as Section 3.4.1 and apply the same post-processing cutoff $T^\star$ to match executed training work across methods.

(1) *MicroGrid* (SC+UPS+TES; pooled-renewable). A conventional micro-grid dispatcher that treats solar and wind as a fully pooled renewable bus (i.e., no source–load pairing in Stage 1). It solves an LP with the same storage models and the same pooling cap $P_{\text{pool}}^{\max,\text{PTP}}$, but without PTP routing constraints.

(2) *CarbonExplorer* (UPS-only compute side). A carbon-aware workload scheduler that defers elastic compute into favorable windows. In our implementation, *CarbonExplorer* operates on the same pooled-renewable abstraction and pooling cap $P_{\text{pool}}^{\max,\text{PTP}}$, with UPS-only storage on the compute side and no TES control; UPS capacity follows the fairness policy above.

(3) *GreenWorks* (UPS-only compute side). A cross-layer framework that uses UPS regulation and load scaling under renewable variability. It uses the same pooled-renewable abstraction and pooling cap $P_{\text{pool}}^{\max,\text{PTP}}$, with UPS-only storage and no TES control.

(4) *NoOptimal* (SC+UPS+TES; heuristic). A per-interval rule-based controller that keeps the same *pair-then-pool* routing and pooling cap $P_{\text{pool}}^{\max,\text{PTP}}$ as *HES4AI* (§3.1), but uses no optimization, no look-ahead, and no training rescheduling ($\alpha(t) \equiv 1$). At each interval, it serves paired IT demand with direct renewables (solar to training, wind to inference) up to demand. Any renewable surplus charges storage with fixed priorities and window flags: wind to SC; solar to TES if in its charge window else to UPS if in its charge window (TES preferred when both are feasible); remaining surplus is curtailed. On a deficit, TES discharges for cooling, SC discharges for inference, and UPS covers the remaining IT shortfall (inference-first), with any residual supplied by the grid via the pooling interface.

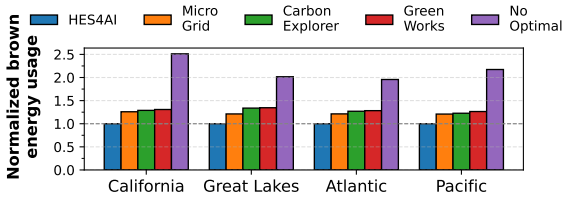
5.3 Overall Comparison with Baselines

Figure 11 compares *HES4AI* against all four baselines in terms of brown energy usage and Green energy wasted. For each region, we normalize each metric to *HES4AI* (thus *HES4AI* is 1.0).

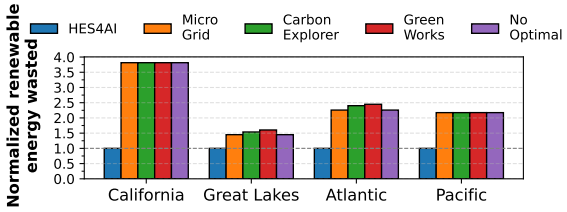
Brown energy usage. Figure 11a shows that *HES4AI* consistently achieves the lowest brown energy consumption across all four regions. *MicroGrid* consumes 1.21×–1.26× as much brown energy as *HES4AI* (1.22× on average). *CarbonExplorer* and *GreenWorks* further increase brown energy usage to about 1.28× and 1.30× on average, respectively. *NoOptimal* performs the worst, consuming 2.17× the brown energy of *HES4AI* on average and up to 2.51× in California.

Green energy wasted. Across regions, *HES4AI* wastes the least renewable energy (Figure 11b). In contrast, *MicroGrid*, *CarbonExplorer*, and *GreenWorks* curtail 2.42×, 2.48×, and 2.51× as much renewable energy on average, respectively. This gap is consistent with the offline facility-sizing result (Figure 7): *MicroGrid*, *CarbonExplorer*, and *GreenWorks* route a larger fraction of renewable power through the shared interface and therefore hit the same cap more frequently, increasing green energy wasted. *NoOptimal* still wastes about 2.42× as much renewable energy while also using the most brown energy overall.

Overall, under the same storage budget and pooling cap $P_{\text{pool}}^{\max,\text{PTP}}$, *HES4AI* achieves both lower grid dependence and higher renewable utilization than pooling-only optimization, UPS-only deferral, or myopic heuristics.

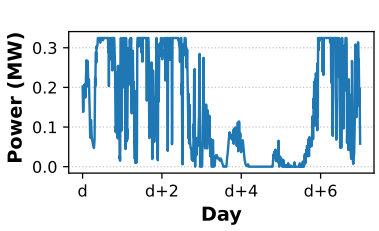


(a) Normalized brown energy usage across regions and methods (lower is better). Each bar is normalized to *HES4AI* in the corresponding region.

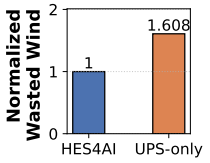


(b) Normalized renewable energy wasted across regions and methods (lower is better). Each bar is normalized to *HES4AI* in the corresponding region.

Figure 11: Comparison of brown energy usage and green energy wasted across four representative regions under five scheduling strategies. All values are normalized to *HES4AI* result in each region (lower is better). The workload traces are derived from [50].



(a) Most fluctuating 7-day wind profile selected from the Pacific Northwest trace.



(b) Normalized wasted wind energy under *PTP* design: Hybrid is 1.0, UPS-only is 1.608.

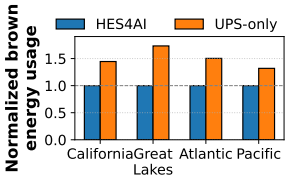
Figure 12: Wind variability and its impact on renewable waste under the same *PTP* design. Left: an example 7-day window with the highest wind fluctuation. Right: wasted wind energy comparison between SC+UPS+TES vs. UPS-only (lower is better), normalized to *Hybrid*.

5.4 Hybrid ESD vs. UPS-only under *PTP* Design.

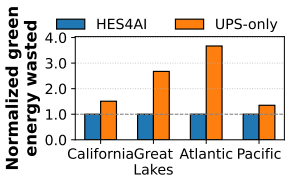
Figure 12 illustrates why adding SC improves *wind* utilization under *PTP* design. Wind generation exhibits rapid, high-frequency surpluses (Figure 12a) that a UPS-only stack cannot absorb effectively due to its limited charging power and the charge-window restriction. In contrast, SC can capture these short bursts without a window constraint, reducing wind wasted. For this 7-days example, *UPS-only* wastes 1.608× more wind energy than *SC+UPS+TES* (Figure 12b).

Figure 13 compares *SC+UPS+TES* and *UPS-only* across four regions, reporting (i) grid (brown) energy and (ii) curtailed renewable (green) energy, both normalized to *SC+UPS+TES* in each region.

Brown energy usage. Across the four regions, *UPS-only* uses between 1.32× (Pacific) and 1.73× (Great Lakes) the brown energy of *HES4AI*, averaging about 1.50× overall. This gap reflects the missing fast and thermal tiers: without SC, the UPS-only design cannot efficiently buffer short, bursty inference transients; without TES, cooling demand must rely more heavily on grid electricity rather than being time-shifted by storing renewable energy as chilled water.



(a) Normalized brown energy usage for *HES4AI* vs. *UPS-only* grouped by different locations.



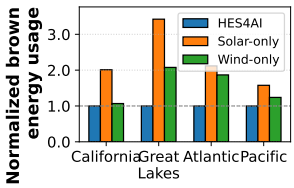
(b) Normalized green energy wasted for *HES4AI* vs. *UPS-only* grouped by different locations.

Figure 13: Comparing brown energy usage (left) and green energy wasted (right) between a hybrid ESDs system and a UPS-only system under the same *PTP* design. All bars are normalized to *HES4AI* in each region (lower is better).

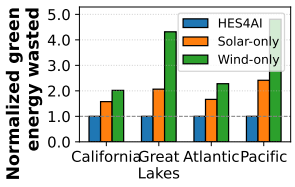
Green energy wasted. This gap is particularly pronounced for wind: with only UPS, fast renewable surpluses are frequently curtailed due to the charge-window restriction and charging power limits. By contrast, leveraging SCs (no window constraint) and TES (large capacity with multi-hour shifting) to absorb surplus energy, substantially reducing wasted under the same storage CapEx.

5.5 Hybrid Renewables vs. Single Source

We compare our default *HES4AI* deployment with both solar and wind against single-source variants (*Solar-only*, *Wind-only*) that use a pooled-renewables dispatch (i.e., *MicroGrid*-style without source–load pairing). All three cases use the same SC+UPS+TES configuration, SoC/power limits, and the same shared pooling interface cap $P_{\text{pool}}^{\max,\text{PTP}}$. In the *Solar-only* and *Wind-only* cases, we reuse the solar and wind traces from the hybrid setting and simply zero out the other source without rescaling. This models a realistic design choice where an operator deploys only solar or only wind at a site; the available renewable supply is therefore intrinsically lower than in the combined solar and wind case unless additional capacity is installed.



(a) Normalized brown energy usage under *HES4AI* vs. single-source renewables.



(b) Normalized green energy wasted under *HES4AI* vs. single-source renewables.

Figure 14: Comparison of brown energy usage and green energy wasted for *HES4AI* vs. single source deployments across four locations. All bars are normalized to *HES4AI* in each region (lower is better).

Brown energy usage. Figure 14a shows that both single-source portfolios consume more brown energy than the hybrid case under

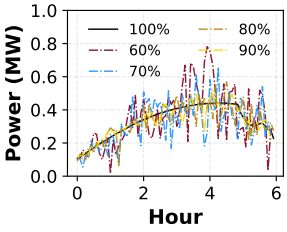
the same storage budget and pooling cap. Averaged across the four locations, *Solar-only* uses about 2.3× the brown energy of the hybrid renewables case (ranging from 1.58× in Pacific to 3.42× in Great Lakes), while *Wind-only* requires about 1.6× as much grid energy (from 1.07× to 2.08×).

Green energy wasted. As shown in Figure 14b, both single-source variants also waste more renewable energy than the hybrid case. *Solar-only* wastes about 1.9× as much renewable energy on average (from 1.58× to 2.41× across regions), and *Wind-only* wastes about 3.4× as much (from 2.02× to 4.81×).

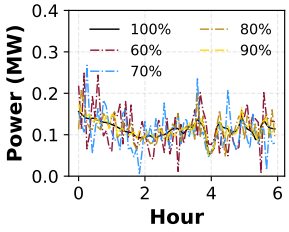
This gap is driven by two factors: (i) losing solar–wind complementarity reduces the effective coverage of deficits (raising grid usage), and (ii) with *Solar-only* or *Wind-only*, there is no source–load pairing, so most renewable power must enter the pooled bus directly, hitting the same $P_{\text{pool}}^{\max}$ more often. Consistent with the offline facility-sizing result (Figure 7), the hybrid renewables case benefits from *PTP* pairing that absorbs most solar on the training side and most wind on the inference side in Stage 1, so only residual power reaches the shared pooling interface. Thus, reducing cap hits and green energy wasted.

5.6 Forecast Accuracy Sensitivity

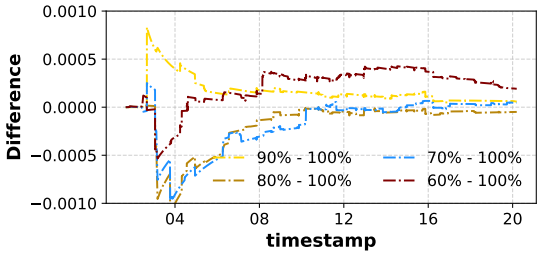
To study robustness to forecast errors, we generate synthetic solar/wind forecasts by perturbing the ground-truth traces and rerun *HES4AI* with identical workloads and ESD configurations. We quantify forecast quality using 1 − MAPE, where MAPE is the *Mean Absolute Percentage Error* computed over the evaluation horizon (with a small ϵ to avoid division by zero when P_t is near zero, e.g., nighttime PV). For each target accuracy level (60–90%), we inject bounded multiplicative noise into each time step and tune the noise magnitude to match the desired 1 − MAPE. We use the renewable traces from California as an example. Figures 15a and 15b show representative perturbed profiles at 5-minute resolution.



(a) Real and noise-perturbed solar generation.



(b) Real and noise-perturbed wind generation.



(c) Cumulative brown-energy difference vs. ideal forecast.

Figure 15: Effect of forecast noise on decisions and brown energy usage.

Figure 15c shows the cumulative net difference in brown energy between noisy forecasts and an ideal (100%) forecast (positive = more grid energy, negative = less). Even at 60% accuracy, the deviation remains very small (on the order of $< 10^{-3}$ in our normalized cumulative metric over a day), indicating that *HES4AI* is robust to moderate forecast noise. This behavior is expected: because *HES4AI* re-optimizes every 5 minutes using the latest measurements and enforces all feasibility constraints with realized renewable availability (Section 3.4.1), any forecast-induced suboptimal choice is quickly corrected in the next interval, with the hybrid ESDs hierarchy buffering the transient impact. As a result, forecast noise has negligible effect on brown energy usage and green energy wasted.

6 Related Work

Carbon-aware operation reduces grid dependence by aligning demand with variable renewables and by using storage. Many systems exploit on-site or procured renewables and adapt workload timing/power to cut emissions [11, 23, 34, 41, 42, 44, 54]. Model-based schedulers time-shift flexible jobs and smooth supply with UPS batteries, but typically operate at hourly granularity and neither couple heterogeneous storage nor distinguish workload types [20, 31, 51].

Hybrid storage has been examined mainly for peak shaving and cost, combining supercapacitors with batteries to leverage complementary power/energy traits [35, 59]. Separately, cooling-side coordination shows thermal energy storage (TES) can shift electric load and reduce peaks [33, 49, 53, 57, 58, 60]. These threads are largely studied in isolation; few systems co-optimize compute and cooling while accounting for storage at different timescales.

Beyond temporal matching at one site, spatial migration places jobs in greener regions. While it can outperform purely temporal shifting in ideal settings, practical headroom, latency SLOs, and policy constraints substantially erode its benefit, and the advantage shrinks as grids decarbonize [43]—underscoring the value of strong local matching.

HES4AI connects these strands: it uses hybrid renewables (solar+wind) with a hybrid storage stack (SC+UPS+TES), explicitly separates training (deferrable) from inference (non-deferrable), and co-optimizes compute and cooling at their natural timescales. This combination addresses the gaps above relative to UPS-centric, deferral-only approaches.

7 Conclusion

We have presented *HES4AI*, a hybrid energy storage framework to minimize the carbon footprint of AI data centers that run ML training and inference on different timescales. Based on the analysis of real-world traces, *HES4AI* pairs solar with elastic training and long-horizon shifting paths (UPS/TES) and pairs wind with non-deferrable inference and fast buffering paths (SC/UPS). This design reduces both brown energy usage and renewable energy wasted under practical device constraints. *HES4AI* is formulated as an online rolling-horizon linear program that jointly decides two-stage routing, hybrid ESD dispatch, and bounded training elasticity. On a hardware testbed, baseline *UPS-only* consumes **1.38–1.42**× as much brown energy and wastes **1.87–1.93**× as much renewable energy as *HES4AI*. In trace-driven simulations across four regions, *HES4AI* consistently outperforms several SOTA baselines: Normalized with *HES4AI*=1.0, these baselines consume **1.21–2.51**× as much brown energy and waste **2.42–2.51**× as much renewable energy.

Acknowledgments

This work was supported, in part, by the U.S. National Science Foundation (NSF) under Grant CNS-2336886. We would also like to thank the anonymous reviewers for their valuable comments.

References

[1] [n. d.]. AI brings soaring emissions for Google and Microsoft, a major contributor to climate change. https://www.npr.org/2024/07/12/g-s1-9545/ai-brings-soaring-emissions-for-google-and-microsoft-a-major-contributor-to-climate-change.

[2] [n. d.]. The Big-Tech Clean Energy Crunch Is Here. https://www.wired.com/story/big-tech-datacenter-energy-power-grid/.

[3] [n. d.]. Googles emissions shot up 48% over five years due to AI. https://www.siliconvalley.com/2024/07/03/googles-emissions-shot-up-48-over-five-years-due-to-ai/.

[4] [n. d.]. Green House Data: Greening the data center. http://www.greenhousedata.com/.

[5] [n. d.]. NVIDIA Announces Financial Results for First Quarter Fiscal 2025. https://nvidianews.nvidia.com/news/nvidia-announces-financial-results-for-first-quarter-fiscal-2025.

[6] [n. d.]. Tech's splurge on AI chips has companies in 'arms race' that's forcing more spending. https://www.cnbc.com/2024/07/25/techs-splurge-on-ai-chips-has-meta-alphabet-tesla-in-arms-race.html.

[7] 2019. Amazon EC2 update – INF1 instances with AWS inferentia chips for high performance cost-effective inferencing | amazon web services. https://aws.amazon.com/blogs/aws/amazon-ec2-update-inf1-instances-with-aws-inferentia-chips-for-high-performance-cost-effective-inferencing/

[8] 2022. AI accelerator - AWS trainium - AWS. https://aws.amazon.com/ai/machine-learning/trainium/

[9] 2024. 2024 Environmental Report - Google Sustainability. https://sustainability.google/reports/google-2024-environmental-report/

[10] 2024. Meta 2024 Sustainability Report. https://sustainability.atmeta.com/2024-sustainability-report/

[11] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon explorer: A holistic framework for designing carbon aware datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 118–132.

[12] Faraz Ahmad and TN Vijaykumar. 2010. Joint optimization of idle and cooling power in data centers while maintaining response time. *ACM Sigplan Notices* 45, 3 (2010), 243–256.

[13] Arnab Choudhury, Yang Wang, Tuomas Pelkonen, Kutta Srinivasan, Abha Jain, Shenghao Lin, Delia David, Siavash Soleimanifard, Michael Chen, Abhishek Yadav, et al. 2024. MAST: Global scheduling of {ML} training across Geo-Distributed datacenters at hyperscale. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 563–580.

[14] Aron P. Dobos. 2014. *PVWatts Version 5 Manual*. Technical Report NREL/TP-6A20-62641. National Renewable Energy Laboratory, Golden, CO, USA. https://www.nrel.gov/docs/fy14osti/62641.pdf

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR* (2021).

[16] Caroline Draxl, Bri-Mathias Hodge, Andrew Clifton, and Jim McCaa. 2015. *Overview and Meteorological Validation of the Wind Integration National Dataset Toolkit*. Technical Report NREL/TP-5000-61740. National Renewable Energy Laboratory, Golden, CO, USA. https://www.nrel.gov/docs/fy15osti/61740.pdf

[17] Caroline Draxl, Bri-Mathias Hodge, Andrew Clifton, and Jim McCaa. 2015. The Wind Integration National Dataset (WIND) Toolkit. *Applied Energy* 151 (2015), 355–366. doi:10.1016/j.apenergy.2015.03.121

[18] Ozan Erdinc, Bulent Vural, and Mehmet Uzunoglu. 2009. A dynamic lithium-ion battery model considering the effects of temperature and capacity fading. In *2009 International Conference on Clean Electrical Power*. IEEE, 383–386.

[19] Wedan Emmanuel Gnibga, Anne Blavette, and Anne-Cécile Orgerie. 2023. Renewable energy in data centers: the dilemma of electrical grid dependency and autonomy costs. *IEEE Transactions on Sustainable Computing* 9, 3 (2023), 315–328.

[20] Íñigo Goiri, William Katsak, Kien Le, Thu D Nguyen, and Ricardo Bianchini. 2013. Parasol and greenswitch: Managing datacenters powered by renewable energy. *ACM SIGPLAN Notices* 48, 4 (2013), 51–64.

[21] Íñigo Goiri, Kien Le, Md E Haque, Ryan Beauchea, Thu D Nguyen, Jordi Guitart, Jordi Torres, and Ricardo Bianchini. 2011. Greenslot: scheduling energy consumption in green datacenters. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11.

[22] Íñigo Goiri, William Katsak, Kien Le, Thu D. Nguyen, and Ricardo Bianchini. 2013. Parasol and GreenSwitch: managing datacenters powered by renewable energy. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[23] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing carbon: The elusive environmental footprint of computing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 854–867.

[24] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. 2018. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 620–629.

[25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385 [cs.CV] https://arxiv.org/abs/1512.03385

[26] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. In *The IEEE International Conference on Computer Vision (ICCV)*.

[27] Jennifer King, Andrew Clifton, and Bri-Mathias Hodge. 2014. *Validation of Power Output for the WIND Toolkit*. Technical Report NREL/TP-5D00-61714. National Renewable Energy Laboratory, Golden, CO, USA. https://www.nrel.gov/docs/fy14osti/61714.pdf

[28] Vasileios Kontorinis, Liuyi Zhang, Baris Aksanli, Jack Sampson, Houman Homayoun, Eddie Pettis, Tajana Rosing, and Dean Tullsen. 2012. Managing Distributed UPS Energy for Effective Power Capping in Data Centers. In *Proceedings of the 39th International Symposium on Computer Architecture (ISCA)*.

[29] Kien Le, Ozlem Bilgir, Ricardo Bianchini, Margaret Martonosi, and Thu D. Nguyen. 2010. Managing the cost, energy consumption, and carbon footprint of internet services. In *Proceedings of the International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*.

[30] Chao Li, Amer Qouneh, and Tao Li. 2012. iSwitch: Coordinating and optimizing renewable energy powered server clusters. *ACM SIGARCH Computer Architecture News* 40, 3 (2012), 512–523.

[31] Chao Li, Rui Wang, Depei Qian, and Tao Li. 2016. Managing server clusters on renewable energy mix. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 11, 1 (2016), 1–24.

[32] Haoran Li, Juan Hou, Tianzhen Hong, Yuemin Ding, and Natasa Nord. 2021. Energy, economic, and environmental analysis of integration of thermal energy storage into district heating systems using waste heat from data centres. *Energy* 219 (2021), 119582.

[33] Li Li, Wenli Zheng, Xiaodong Wang, and Xiaorui Wang. 2014. Coordinating liquid and free air cooling with workload allocation for data center power minimization. In *11th international conference on autonomic computing (ICAC 14)*. 249–259.

[34] Yunbo Li, Anne-Cécile Orgerie, and Jean-Marc Menaud. 2017. Balancing the use of batteries and opportunistic scheduling policies for maximizing renewable energy consumption in a cloud data center. In *2017 25th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*. IEEE, 408–415.

[35] Longjun Liu, Chao Li, Hongbin Sun, Yang Hu, Juncheng Gu, Tao Li, Jingmin Xin, and Nanning Zheng. 2016. HEB: Deploying and managing hybrid energy buffers for improving datacenter efficiency and economy. In *Proceedings of the 42nd International Symposium on Computer Architecture (ISCA)*.

[36] Kendall Mongird, Vilayanur Viswanathan, Jan Alam, Charlie Vartanian, and Vincent Sprenkle. 2020. https://www.pnnl.gov/sites/default/files/media/file/LeadAcid_Methodology.pdf

[37] National Renewable Energy Laboratory. 2012. *Development of the Wind Turbine Models for the Western Wind and Solar Integration Study*. Technical Report NREL/SR-5500-52780. National Renewable Energy Laboratory, Golden, CO, USA. https://www.nrel.gov/docs/fy12osti/52780.pdf

[38] Daniel Richins, Dharmisha Doshi, Matthew Blackmore, Aswathy Thulaseedharan Nair, Neha Pathapati, Ankit Patel, Brainard Daguman, Daniel Dobrijalowski, Ramesh Illikkal, Kevin Long, et al. 2020. Missing the forest for the trees: End-to-end ai application performance in edge data centers. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 515–528.

[39] Pranoy Roy, Yuan Liao, and JiangBiao He. 2022. Economic dispatch for grid-connected wind power with battery-supercapacitor hybrid energy storage system. *IEEE Transactions on Industry Applications* 59, 1 (2022), 1118–1128.

[40] Manajit Sengupta, Aron Habte, Grant Buster, Michael Rossol, Yu Xie, and Marissa Foster. 2020. A Status Update on the National Solar Radiation Data Base (NSRDB). PV Performance Modeling Collaborative (PVPMC) Webinar. https://docs.nrel.gov/docs/fy20osti/77121.pdf PVPMC Webinar on Solar Resource Assessment, June 24, 2020.

[41] Navin Sharma, Sean Barker, David Irwin, and Prashant Shenoy. 2011. Blink: managing server clusters on intermittent power. In *Proceedings of the sixteenth international conference on Architectural support for programming languages and operating systems*. 185–198.

[42] Haiying Shen, Haoyu Wang, Jiechao Gao, and Rajkumar Buyya. 2023. An instability-resilient renewable energy allocation system for a cloud datacenter. *IEEE Transactions on Parallel and Distributed Systems* 34, 3 (2023), 1020–1034.

[43] Thanathorn Sukprasert, Abel Souza, Noman Bashir, David Irwin, and Prashant Shenoy. 2024. On the limitations of carbon-aware temporal and spatial workload shifting in the cloud. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 924–941.

[44] Jinghan Sun, Zibo Gong, Anup Agarwal, Shadi Noghabi, Ranveer Chandra, Marc Snir, and Jian Huang. 2024. Exploring the efficiency of renewable energy-based modular data centers at scale. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 552–569.

[45] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.

[46] P. Thibodeau. 2008. Wind power data center project planned in urban area. http://www.computerworld.com/.

[47] Di Wang, Chuangang Ren, Anand Sivasubramaniam, Bhuvan Urgaonkar, and Hosam Fathy. 2012. Energy storage in datacenters: what, where, and how much?. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*. 187–198.

[48] Richard Wang, Shu-Chien Hsu, Saina Zheng, Jieh-Haur Chen, and Xuran Ivan Li. 2020. Renewable energy microgrids: Economic evaluation and decision making for government policies to contribute to affordable and clean energy. *Applied Energy* 274 (2020), 115287.

[49] Yefu Wang, Xiaorui Wang, and Yanwei Zhang. 2011. Leveraging Thermal Storage to Cut the Electricity Bill for Datacenter Cooling. In *Proceedings of the 4th Workshop on Power-Aware Computing and Systems (HotPower)*.

[50] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters. In *19th USENIX symposium on networked systems design and implementation (NSDI 22)*. 945–960.

[51] Ting Yang, Yucheng Hou, Young Choon Lee, Hao Ji, and Albert Y Zomaya. 2020. Power control framework for green data centers. *IEEE Transactions on Cloud Computing* 10, 4 (2020), 2876–2886.

[52] Jeff Zhang, Sameh Elnikety, Shuayb Zarar, Atul Gupta, and Siddharth Garg. 2020. Model-Switching: Dealing with Fluctuating Workloads in Machine-Learning-as-a-Service Systems. In *Proceedings of the 12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.

[53] Yanwei Zhang, Yefu Wang, and Xiaorui Wang. 2011. Capping the Electricity Cost of Cloud-Scale Data Centers with Impacts on Power Markets. In *Proceedings of the 20th ACM International Symposium on High Performance Distributed Computing (HPDC)*.

[54] Yanwei Zhang, Yefu Wang, and Xiaorui Wang. 2011. Greenware: Greening cloud-scale data centers to maximize the use of renewable energy. In *ACM/IFIP/USENIX international conference on distributed systems platforms and open distributed processing*. Springer, 143–164.

[55] Yanwei Zhang, Yefu Wang, and Xiaorui Wang. 2012. TEStore: Exploiting Thermal and Energy Storage to Cut the Electricity Bill for Datacenter Cooling. In *Proceedings of the 8th International Conference on Network and Service Management (CNSM)*.

[56] Dongyu Zheng, Lei Liu, Guoming Tang, Yi Wang, and Weichao Li. 2023. Power demand reshaping using energy storage for distributed edge clouds. *IEEE Transactions on Parallel and Distributed Systems* 35, 2 (2023), 362–376.

[57] Wenli Zheng, Kai Ma, and Xiaorui Wang. 2014. Exploiting thermal energy storage to reduce data center capital and operating expenses. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 132–141.

[58] Wenli Zheng, Kai Ma, and Xiaorui Wang. 2015. TE-Shave: Reducing data center capital and operating expenses with thermal energy storage. *IEEE Trans. Comput.* 64, 11 (2015), 3278–3292.

[59] Wenli Zheng, Kai Ma, and Xiaorui Wang. 2017. Hybrid Energy Storage with Supercapacitor for Cost-Efficient Data Center Power Shaving and Capping. *IEEE Transactions on Parallel and Distributed Systems* 28, 4 (2017), 1105–1118.

[60] Wenli Zheng and Xiaorui Wang. 2015. Data Center Sprinting: Enabling Computational Sprinting at the Data Center Level. In *Proceedings of the 35th IEEE International Conference on Distributed Computing Systems (ICDCS)*.